

Fiche de révision — Data Science & Machine Learning

Tout comprendre simplement, avec des exemples concrets — pour être fin prêt en tant que Data Scientist Junior

Cette fiche reprend **tous les sujets du test** que tu as vu (apprentissage, algorithmes, évaluation, pièges, préparation des données, Python). Chaque notion est expliquée **comme à un enfant de 8 ans**, puis illustrée par un exemple concret, et résumée par un point « à retenir ». À la fin, tu trouveras un **mémo express** qui répond en une ligne à chacune des 19 questions.

Code couleur : **bleu** = l'image simple • **vert** = exemple concret • **orange** = à retenir • **rouge** = piège à éviter.

0. C'est quoi le Machine Learning (ML) ?

Le **Machine Learning** (apprentissage automatique), c'est apprendre à un ordinateur à faire une tâche à **partir d'exemples**, au lieu de lui écrire toutes les règles à la main.

L'image simple : Tu n'expliques pas à un enfant toutes les règles pour reconnaître un chat. Tu lui montres beaucoup de chats. Au bout d'un moment, il reconnaît un chat qu'il n'a jamais vu. Le ML fait pareil avec des données.

Deux grandes familles de tâches reviennent tout le temps : la **classification** (ranger dans des catégories : spam / pas spam) et la **régression** (prédire un nombre : le prix d'une maison).

1. Les 3 types d'apprentissage (Q1)

Apprentissage supervisé

On apprend à partir de données **étiquetées** : pour chaque exemple, on connaît déjà la bonne réponse (X = les caractéristiques, y = la réponse).

L'image simple : Un cahier avec les corrigés. On te montre des photos en disant « ça, c'est un chat », « ça, c'est un chien ». Ensuite tu sais trier tout seul.

Exemple concret : Prédire si un e-mail est un spam, à partir de milliers d'e-mails déjà marqués « spam » ou « normal ».

Apprentissage non supervisé

Il n'y a **pas de réponse** donnée. L'algorithme cherche tout seul une structure cachée : des groupes (clustering) ou des axes qui résument l'information (réduction de dimension).

L'image simple : On te donne un grand sac de billes mélangées sans rien te dire, et tu fais des tas qui se ressemblent (par couleur, par taille). Personne ne t'a donné les catégories à l'avance.

Exemple concret : Regrouper des clients en profils qui se ressemblent (« petits acheteurs réguliers », « gros acheteurs occasionnels ») sans savoir les groupes au départ.

Apprentissage semi-supervisé

Un mélange : **quelques** données sont étiquetées, et **beaucoup** ne le sont pas. On utilise les deux. Très utile quand étiqueter coûte cher (il faut un humain).

L'image simple : Sur ton sac de billes, on t'a étiqueté seulement 5 billes (« celle-ci est rare »). Tu te sers de ces 5 indices pour deviner pour toutes les autres.

À retenir : Supervisé = **avec** les réponses. Non supervisé = **sans** les réponses. Semi-supervisé = **un peu** de réponses + beaucoup sans.

2. Les algorithmes à connaître (Q2, Q3, Q4)

2.1 Classification supervisée (Q2)

Régression logistique. Malgré son nom, c'est un classifieur. Elle calcule une **probabilité** (entre 0 et 1) d'appartenir à une classe, puis décide avec un seuil (souvent 0,5).

L'image simple : Une machine qui répond « il y a 80 % de chances que ce soit un chat ». Au-dessus de 50 %, elle dit « chat ».

Arbre de décision / Forêt aléatoire. L'arbre pose une suite de questions oui/non sur les variables pour séparer les classes. La **forêt aléatoire** fait voter beaucoup d'arbres différents et garde la réponse majoritaire (plus stable).

L'image simple : L'arbre, c'est le jeu des « 20 questions » (A-t-il des moustaches ? Fait-il du bruit ?...). La forêt, c'est demander à 100 amis qui jouent chacun aux 20 questions, puis suivre l'avis de la majorité.

Exemple concret : Accepter ou refuser un prêt à partir de l'âge, du salaire, de l'historique. La forêt aléatoire dit aussi quelles variables comptent le plus.

SVM (machine à vecteurs de support). Elle trace la frontière qui sépare les classes en laissant la **plus grande marge** possible (le plus grand couloir vide) entre les deux groupes. Avec des « noyaux », elle gère aussi les frontières courbées.

L'image simple : Tu sépares les filles et les garçons dans la cour en traçant une ligne au sol, en laissant le plus d'espace possible de chaque côté.

KNN (k plus proches voisins). Pour classer un nouveau point, on regarde ses **k voisins les plus proches** et on prend la classe majoritaire. Il n'y a presque pas d'apprentissage : tout se joue au moment de prédire.

2.2 Classification non supervisée / clustering (Q3)

K-means. On choisit k groupes. On place k « centres », on rattache chaque point au centre le plus proche, on recalcule les centres, et on recommence jusqu'à ce que ça ne bouge plus.

L'image simple : Tu ranges tes jouets dans k boîtes : chaque jouet va dans la boîte dont le « jouet-type » au centre lui ressemble le plus.

Clustering hiérarchique (CAH). On construit un **arbre de regroupements** (dendrogramme) : on fusionne petit à petit les groupes les plus proches, puis on « coupe » l'arbre au niveau voulu.

DBSCAN. Regroupe par **densité** : il relie les zones où les points sont serrés et laisse de côté les points isolés (le « bruit »). Pas besoin de fixer le nombre de groupes à l'avance.

Exemple concret : Dans une cour de récré, DBSCAN repère les grappes d'enfants serrés (des groupes de jeu) et ignore l'enfant tout seul dans un coin (un point aberrant).

2.3 Régression (Q4)

Régression linéaire. On trace la **droite** (ou le plan) qui passe le mieux au milieu des points, en rendant les erreurs les plus petites possibles.

L'image simple : Tu as des points « taille → poids » sur une feuille. Tu traces la règle droite qui passe au plus près de tous les points pour deviner le poids d'une nouvelle personne.

Ridge et Lasso (régressions régularisées). La même idée, mais on ajoute une **pénalité** pour empêcher les coefficients de devenir énormes. Le Lasso peut même mettre des coefficients à zéro : il fait le tri des variables.

Forêt aléatoire / Gradient Boosting de régression. Des ensembles d'arbres adaptés aux nombres. Ils captent très bien les relations **non linéaires** (quand ce n'est pas une simple droite).

À retenir : Classification = prédire une **catégorie**. Régression = prédire un **nombre**. Clustering = **regrouper sans étiquettes**.

3. Évaluer un modèle (Q5)

3.1 La matrice de confusion (classification)

C'est un tableau qui compare ce que le modèle a prédit avec la vérité. Quatre cases :

VP (Vrai Positif) = malade détecté malade. **FP** (Faux Positif) = sain dit malade (fausse alerte). **FN** (Faux Négatif) = malade dit sain (on l'a raté). **VN** (Vrai Négatif) = sain dit sain.

Exemple concret : Test médical sur 100 personnes, dont 10 vraiment malades. Le modèle détecte 8 vrais malades (VP), en rate 2 (FN), donne 4 fausses alertes (FP) et reconnaît 86 personnes saines (VN).

	Réalité : Malade	Réalité : Sain
Prédit : Malade	VP = 8	FP = 4
Prédit : Sain	FN = 2	VN = 86

Accuracy (exactitude) = $(VP+VN) / \text{total} = (8+86)/100 = 94 \%$. La part de bonnes réponses.

Précision = $VP / (VP+FP) = 8/12 = 66,7 \%$. Quand le modèle dit « malade », a-t-il raison ?

Rappel (Recall) = $VP / (VP+FN) = 8/10 = 80 \%$. Parmi les vrais malades, combien sont attrapés ?

F1-score = moyenne équilibrée de la précision et du rappel = **72,7 %**. Et la **courbe ROC / AUC** mesure la qualité globale du tri.

Piège : L'accuracy peut **mentir** quand les classes sont déséquilibrées (voir Q9). Un modèle qui dit toujours « sain » aurait 90 % d'accuracy ici... mais raterait tous les malades.

3.2 Évaluer une régression

MAE = erreur moyenne en valeur absolue. **MSE / RMSE** = erreur quadratique (punit fort les grosses erreurs). **R²** = part de la réalité expliquée par le modèle (1 = parfait, 0 = nul).

Exemple concret : On prédit le prix de 3 maisons (en milliers d'€). Réel : 200, 250, 300. Prédit : 210, 230, 305. Erreurs : +10, -20, +5.

$MAE = (10+20+5)/3 = 11,7$ • $MSE = (100+400+25)/3 = 175$ • $RMSE = \sqrt{175} = 13,2$ (mille €).

3.3 Évaluer un clustering (non supervisé)

Comme il n'y a pas de « bonne réponse », on mesure la qualité des groupes : le **score de silhouette** (les points sont-ils bien dans leur groupe ?), la **méthode du coude** (inertie) pour choisir le nombre de groupes k, et l'indice de **Davies-Bouldin**.

4. Les grands pièges (Q6 à Q12)

4.1 Surapprentissage (overfitting) et sous-apprentissage (Q6, Q7)

Le **surapprentissage** : le modèle apprend le bruit et les détails par cœur, marche très bien sur les données d'entraînement mais **rate** sur les nouvelles. Le **sous-apprentissage** : il est trop simple et se trompe partout.

L'image simple : L'élève qui apprend le livre par cœur (overfitting) : dès qu'on change un mot dans la question, il est perdu. L'élève qui n'a pas révisé (underfitting) se trompe sur tout. On veut celui qui a compris : ni trop, ni trop peu.

Comment l'éviter (régression ET classification) : validation croisée, **régularisation** (Ridge/Lasso), simplifier le modèle (moins de variables, arbres moins profonds), récupérer plus de données. En plus : élaguer les arbres ; pour les réseaux de neurones, **dropout** et **early stopping** ; et les méthodes d'ensemble (forêt aléatoire).

À retenir : Overfitting = par cœur sans comprendre. La parade n°1 : tester sur des données que le modèle n'a jamais vues (validation croisée).

4.2 KNN : régler le nombre de voisins k (Q8)

Pour réduire le surapprentissage avec KNN, il faut **augmenter k**.

L'image simple : $k = 1$: tu copies juste ton voisin le plus proche, même s'il s'est trompé (tu sur-apprends le bruit). k plus grand : tu demandes l'avis de plusieurs voisins et tu suis la majorité, c'est plus prudent et plus lisse. Mais k trop grand = tu demandes à toute l'école, ça devient flou (sous-apprentissage).

À retenir : k petit → surapprentissage. k grand → décision lissée. On choisit le bon k par **validation croisée**.

4.3 Quelle métrique optimiser ? Faux positifs vs faux négatifs (Q9, Q11)

Tout dépend du **coût de l'erreur**. Pour une **détection de bombes**, il faut optimiser le **Rappel (Recall)** : rater une vraie bombe (faux négatif) est catastrophique, donc on veut n'en manquer aucune, quitte à avoir des fausses alertes.

L'image simple : Un faux négatif = on a raté le danger. Un faux positif = fausse alerte. Pour une bombe, mieux vaut 10 fausses alertes que 1 bombe ratée.

Règle générale : quand rater un cas positif est grave (maladie grave, fraude, bombe) → on accepte **plus de faux positifs**. Quand accuser à tort coûte cher (condamner un innocent, bloquer un bon client, supprimer un e-mail important) → on accepte **plus de faux négatifs**. On règle ça avec le **seuil de décision**.

Piège : Quand une classe est très rare (bombes, fraudes), l'accuracy n'est pas la bonne mesure : regarde plutôt le rappel et la précision.

4.4 La malédiction de la dimensionnalité (Q10)

Plus on ajoute de variables (dimensions), plus l'espace devient **immense et vide** : les points s'éloignent tous les uns des autres et les distances perdent leur sens. Les algorithmes basés sur la distance (KNN, k-means) en souffrent, et il faut énormément de données.

L'image simple : Retrouver un ami : facile dans un couloir (1 dimension), plus dur dans un immeuble (2-3 dimensions), impossible dans un monde à 100 dimensions où tout le monde est « loin ».

Comment l'éviter : réduction de dimension (**ACP / PCA**), sélection des variables vraiment utiles, régularisation, et plus de données.

4.5 Les valeurs aberrantes / outliers (Q12)

Une valeur aberrante **tire le modèle vers elle** : elle déplace la frontière de décision, fausse les moyennes et écarts-types (utilisés pour normaliser), et trompe surtout la régression logistique, le SVM et le KNN. Souvent c'est une erreur de saisie, et le modèle apprend une fausse règle.

L'image simple : Un homme de 2,50 m dans un groupe : la « taille moyenne » devient fausse et ne représente plus personne.

Que faire : les repérer (boxplot, z-score, écart interquartile) puis les corriger, les retirer, ou utiliser des méthodes robustes.

5. Préparer les données (Q13, Q14, Q15)

5.1 Mettre à la même échelle (Q13)

Quand les variables ont des **unités et des ordres de grandeur différents**, on les remet sur la même règle : **normalisation Min-Max** (tout entre 0 et 1) ou **standardisation z-score** (moyenne 0, écart-type 1).

L'image simple : Comparer un salaire (en milliers) et un âge (en dizaines) sans mise à l'échelle, c'est comme comparer des kilomètres et des millimètres : le grand nombre écrase le petit. On met tout sur la même règle graduée.

À retenir : Indispensable pour les algos à base de distance (KNN, k-means, SVM) et la descente de gradient.

5.2 Gérer les données manquantes (Q14)

D'abord regarder **combien** il en manque et **pourquoi**. Ensuite : **supprimer** les lignes/colonnes si c'est marginal ; **imputer** (remplir) avec la médiane, la moyenne, le mode, ou plus finement (KNN, régression) ; ajouter un **indicateur** « **valeur manquante** » ; ou utiliser un modèle qui gère les trous tout seul.

L'image simple : Un questionnaire avec des cases vides : soit tu jettes la feuille, soit tu remplis la case avec la réponse la plus typique, soit tu notes « non répondu ».

5.3 Visualiser la variable cible / target (Q15)

Si la cible est une **catégorie** : un **diagramme en barres** pour voir la répartition des classes (et repérer un déséquilibre), et des **boxplots** par classe. Si la cible est un **nombre** : un **histogramme** pour sa distribution, des **nuages de points** avec les variables, et une **heatmap de corrélation**.

6. Python pour la Data Science (Q16, Q17, Q18, Q19)

6.1 lambda, map et filter (Q18)

lambda = une mini-fonction sans nom. **filter** = une passoire qui garde ce qui passe un test. **map** = une machine qui transforme chaque élément.

L'image simple : filter = un tamis qui ne laisse passer que les billes paires. map = une usine qui peint chaque bille qui passe. À la fin, list() vide le tapis roulant dans un panier.

Énoncé : créer la liste des carrés des nombres pairs de [1..10]. On garde les pairs avec filter, on les met au carré avec map :

```
liste = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evenNumbers = list(map(lambda x: x**2,
                       filter(lambda x: x % 2 == 0, liste)))
print(evenNumbers) # [4, 16, 36, 64, 100]
```

Détail : **x % 2 == 0** veut dire « x est pair » (reste nul quand on divise par 2). **x**2** veut dire « x au carré ».

6.2 Le problème « Two Sum » (Q19)

On cherche deux nombres d'une liste dont la **somme vaut une cible**, et on renvoie leurs positions (indices). L'astuce : parcourir la liste une seule fois en notant ce qu'on a déjà vu dans un dictionnaire ; pour chaque nombre, on cherche son « complément » (cible - nombre).

L'image simple : Tu veux deux cartes qui font 9. Tu as une carte « 2 ». Tu cherches dans ton carnet si tu as déjà vu un « 7 ». Sinon, tu notes « 2 » dans le carnet et tu continues.

```
def two_sum(nums, target):
    vus = {} # valeur -> indice
    for i, n in enumerate(nums):
        complement = target - n
        if complement in vus:
            return [vus[complement], i]
        vus[n] = i
```

```
two_sum([2, 7, 11, 15], 9) # [0, 1]
two_sum([3, 2, 4], 6) # [1, 2]
```

Pourquoi un dictionnaire ? Parce que « chercher si j'ai déjà vu X » y est **instantané**. On parcourt la liste une seule fois : c'est rapide (**O(n)**).

6.3 Langage et bibliothèques (Q16, Q17)

Réponse type, à adapter à ton goût : le langage préféré est **Python** (simple et riche). Bibliothèques de base : **pandas** et **NumPy** (manipuler les données), **scikit-learn** (machine learning), **Matplotlib** et **Seaborn** (graphiques), **statsmodels** (statistiques). À côté : **SQL** (interroger les bases) et **Power BI** (restitution).

Pour « ton algorithme préféré » (Q16), choisis-en un et explique **pourquoi**. Exemple : la **forêt aléatoire**, parce qu'elle est robuste, demande peu de réglages et indique quelles variables comptent le plus — pratique pour expliquer un résultat.

7. Le déroulé d'un projet (pour penser comme un Data Scientist)

Au-delà des définitions, un junior doit connaître l'**ordre des étapes** d'un projet :

1) Comprendre le problème → **2) Récupérer** les données → **3) Nettoyer** (manquants, aberrants) → **4) Explorer** (graphes, corrélations) → **5) Préparer** (mise à l'échelle, découpe **train / test**) → **6) Entraîner** un modèle → **7) Évaluer** (sur les données de test) → **8) Améliorer** → **9) Déployer**.

L'image simple : On entraîne sur les « exercices avec corrigé » (train) et on note l'élève sur un « examen jamais vu » (test). Sinon, c'est comme donner le sujet de l'examen à l'avance : la note ne veut plus rien dire.

À retenir : Ne JAMAIS évaluer un modèle sur les données qui ont servi à l'entraîner. On garde toujours un jeu de **test** à part, et on utilise la **validation croisée**.

8. Mémo express — les 19 questions en une ligne

#	Réponse en une ligne
1	Supervisé = avec réponses ; non supervisé = sans ; semi = un peu de réponses + beaucoup sans.
2	Classif supervisée : régression logistique, forêt aléatoire, SVM (+ KNN).
3	Clustering : K-means, hiérarchique (CAH), DBSCAN.
4	Régression : linéaire, Ridge/Lasso, forêt aléatoire / gradient boosting.
5	Classif : accuracy, précision/rappel/F1, ROC-AUC. Régression : MAE, RMSE, R ² . Clustering : silhouette, coude, Davies-Bouldin.
6	Anti-overfitting régression : régularisation, validation croisée, simplifier, plus de données.

#	Réponse en une ligne
7	Anti-overfitting classif : validation croisée, élagage des arbres, ensembles, dropout/early stopping.
8	KNN : augmenter k pour réduire le surapprentissage (k choisi par validation croisée).
9	Détection de bombes : optimiser le Recall (ne rater aucun vrai positif).
10	Dimensionnalité : trop de variables = espace vide ; éviter via ACP, sélection, régularisation.
11	FP vs FN : selon le coût de l'erreur ; on règle le seuil de décision.
12	Outliers : faussent les stats et tordent le modèle ; les repérer et les traiter.
13	Échelles différentes : normaliser (Min-Max) ou standardiser (z-score).
14	Manquants : supprimer, imputer (médiane/KNN), ou marquer ; selon volume et cause.
15	Cible catégorielle : barres + boxplots. Cible continue : histogramme, scatter, corrélations.
16	Algo préféré : en choisir un et justifier (ex. forêt aléatoire = robuste + importance des variables).
17	Python + pandas, NumPy, scikit-learn, Matplotlib/Seaborn, statsmodels ; SQL et Power BI à côté.
18	<code>list(map(lambda x: x**2, filter(lambda x: x%2==0, liste)))</code> -> [4,16,36,64,100].
19	Two Sum : un dictionnaire des valeurs vues, chercher le complément ; O(n).

Tu as tout : relis le mémo la veille, et entraîne-toi à réécrire le code des Q18 et Q19 sans regarder. Bon courage !